

Rung 4 — Make Lean check it (the first real cliff)

Where you start: you can now derive our certificate by hand, composition and all (Rungs 1–3). But "by hand" means *you* checked it — and humans drop signs. This rung hands the proof to a machine that won't.

What you'll be able to do: read a Lean theorem statement, understand what "machine-checked" and "sorryAx-free" actually guarantee, and write your first small proof yourself.

Honesty up front: this is a genuine cliff. Rungs 1–3 needed Calc II; this one is a new *tool* with its own learning curve, like picking up a new language and IDE at once. Budget weeks, not an evening, and lean hard on the Natural Number Game. That's normal — the track told you the cliffs were coming.

1. The bench

Every certificate in this project ends with the same two words in its evidence: **sorryAx-free**. That's the strongest claim we make, and it's worth knowing exactly what it means — because it's the difference between "we're pretty sure" and "a computer verified there are no gaps."

```
MachLib.Real.eml_sin_full_fwd_error :
  ∀ x, 0 ≤ x → x ≤ 1 → |eml_sin_rtl x - Real.sin x| ≤ 18/D + x^6
```

That reads exactly like the $\forall$ statement from Rung 1.5 — "for all x between 0 and 1, ...". The question this rung answers: what does it mean for a **computer** to have *proven* that line?

2. The one new idea: a proof is a program (propositions-as-types)

Here's the beautiful trick Lean is built on, called **propositions-as-types** (or Curry–Howard):

- A **theorem** is a *type* (like `int` or `list`, but fancier — e.g. "the type of proofs that A implies B").
- A **proof** is a *value* of that type — a little program.
- Checking a proof = **type-checking** that program. If it type-checks, the proof has no gaps. If it doesn't, Lean rejects it and points at the hole.

So "machine-checked" isn't a computer *searching* for a proof — *you* write the proof, and the computer mechanically verifies each step connects. It's an extremely strict compiler for arguments.

sorry is Lean's escape hatch: it means "trust me, I'll fill this in later," and it makes the proof compile with a hole. **sorryAx-free** means the finished proof contains **no sorry** and leans on **no extra unproven axioms** — every step, all the way down to the logical foundations and the one honest `floor` axiom, is checked. `take build` passing = the whole thing verified. That's the guarantee behind our `claim_flags`.

3. Do it by hand (yes, really)

You start with toy proofs, and it feels like a puzzle game — because it is. Here's the shape of a first Lean proof (proving `n + 0 = n` -flavored facts is literally lesson 1 of the Natural Number Game):

```
theorem add_zero (n : Nat) : n + 0 = n := by
  rfl      -- "by reflexivity": both sides reduce to the same thing
```

And a two-step one, to feel the "tactics" that drive a proof:

```
theorem succ_inj (a b : Nat) (h : a + 1 = b + 1) : a = b := by
  injection h -- peel the +1 off both sides
```

You don't learn this by reading — you learn it by *playing the Natural Number Game* (in your browser, no install), where you re-prove arithmetic from nothing, one tactic at a time. That's the on-ramp. `Mathematics in Lean` and `Theorem Proving in Lean 4` (both free) are the real references once the game clicks.

4. Make it formal — the honest floor of the tower

Every proof bottoms out somewhere. Ours bottoms out at:

- Lean's **kernel** (a tiny, heavily-audited logic checker — the thing you ultimately trust),
- **Mathlib** (the community's giant library of already-proven math: `Real.sin`, the MVT, the triangle inequality — all the tools you used by hand on Rungs 2–3, already formalized),
- and **one** honest domain axiom: that `floor` truncates by less than 1 (the Rung 1 fact).

`sorryAx` -free means: nothing else sneaks in. No "assume the hard part." That discipline is the same one you've seen on the hardware side — `candidate_only`, "no claim marked true until it's actually observed." Same honesty, top to bottom.

5. Watch it hold

The "capture" for this rung isn't a scope — it's the build log. When `machlib` runs `lake build` and it finishes with no errors and no `sorry`, a computer has just re-checked every step of the argument you learned to make on Rungs 1–3. That green build is the evidence. It's why we can put a proven number in front of silicon and mean it.

The cliffhanger → Rung 5

Everything so far bounds the error of **one** computation — a single `x` through the pipeline. But real systems don't compute once; they **run over time**. A control loop feeds its output back in, again and again. A one-shot error bound says nothing about whether the loop stays safe on step number ten thousand. Bounding a system *forever* needs a different, gorgeous idea: an "energy" that can only go downhill.

→ **Next:** Rung 5 — *Why the loop can't escape* — dynamics, invariant sets, and Lyapunov functions.

Exercises & checkpoint

1. Play the **Natural Number Game** (browser, no install), World 1. Just finish the tutorial and the first addition level — feel the `rf1` and `rw` tactics.
2. In your own words: what is the difference between a proof containing `sorry` and a `sorryAx`-free proof? Which one do our evidence packets require?
3. One sentence: what does "propositions-as-types" mean — what is a theorem, and what is a proof?

Checkpoint: you've closed at least World 1 of the Natural Number Game, and you can say what "machine-checked" actually guarantees.